

Java Multithreading & Concurrency Deep Dive Reference

Java Multithreading & Concurrency — Deep Dive Reference

1. Thread Lifecycle — Deeper Theory
 2. Runnable vs. Callable — Deeper Theory
 3. `synchronized` — Deeper Theory
 4. `wait()` / `notify()` — Deeper Theory
 5. `volatile` — Deeper Theory (Java Memory Model)
 6. Thread Safety — Consolidated Theory
 7. Executor Framework — Deeper Theory
 8. Thread Pools — Sizing Theory
 9. `CompletableFuture` — Full Treatment (article only namechecks this)
 10. `java.util.concurrent` — Locks, Atomics, `ConcurrentHashMap`
 11. Deadlocks, Livelocks, and Starvation
 12. Coding Questions
- Quick-Reference Summary Table

Java Multithreading & Concurrency — Deep Dive Reference

A companion to the DigitalOcean multithreading article, filling in theory the original left thin (CompletableFuture, atomics, ConcurrentHashMap internals, livelocks) and going deeper on everything else. Includes a coding question bank with answers at the end.

1. Thread Lifecycle — Deeper Theory

Java's `Thread.State` enum actually defines **six** states, not five — the article's "Running" is a conceptual sub-state of `RUNNABLE`, not a distinct JVM state:

```
public enum State {  
    NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED  
}
```

JVM State	Meaning
NEW	Thread object created, <code>start()</code> not yet called
RUNNABLE	Eligible to run; may be actually executing or just waiting for CPU time (OS scheduler decides)
BLOCKED	Waiting to acquire a monitor lock (stuck on <code>synchronized</code> entry)
WAITING	Waiting indefinitely — via <code>Object.wait()</code> (no timeout), <code>Thread.join()</code> (no timeout), or <code>LockSupport.park()</code>
TIMED_WAITING	Same as above but with a timeout — <code>Thread.sleep(ms)</code> , <code>wait(ms)</code> , <code>join(ms)</code> , <code>LockSupport.parkNanos()</code>
TERMINATED	<code>run()</code> returned or threw an uncaught exception

Key nuance the article glosses over: `BLOCKED` and `WAITING` are *not the same thing*, and this is a common interview trip-up: - `BLOCKED` = actively trying to enter a `synchronized` block/method but another thread holds the lock. - `WAITING` = the thread *voluntarily gave up* the lock (via `wait()`) and is waiting to be notified.

You can inspect a live thread's state programmatically:

```
Thread t = new Thread(() -> { /* ... */ });  
t.start();  
System.out.println(t.getState()); // RUNNABLE, WAITING, etc.
```

Thread death is one-way. Once `TERMINATED`, calling `start()` again throws `IllegalThreadStateException`. If you need to "restart" work, submit a new task to a pool — don't try to reuse a `Thread` object.

2. Runnable vs. Callable — Deeper Theory

The article covers `Runnable` well but under-explains `Callable`. Here's the fuller contrast:

	<code>Runnable</code>	<code>Callable</code>
Method	<code>void run()</code>	<code>V call()</code> throws <code>Exception</code>
Return value	None	Yes, generic type <code>V</code>
Checked exceptions	Cannot declare/throw	Can throw any checked exception
Introduced	Java 1.0	Java 5 (<code>java.util.concurrent</code>)
Used directly with <code>Thread</code> ?	Yes (<code>new Thread(runnable)</code>)	No — only via <code>ExecutorService.submit()</code>
Result retrieval	N/A	Via <code>Future<V></code>

Why `Callable` exists at all: `Runnable.run()` was locked into the `void /no-checked-exception` contract before Java had generics or a task-result abstraction. Retrofitting it would break binary compatibility, so Java 5 added a parallel interface instead.

```
Callable<Integer> task = () -> {
    Thread.sleep(500);
    return 42;
};

ExecutorService ex = Executors.newSingleThreadExecutor();
Future<Integer> future = ex.submit(task);
Integer result = future.get(); // blocks until done, or throws
ExecutionException
ex.shutdown();
```

Future API worth knowing beyond `get()`: - `future.get(long timeout, TimeUnit unit)` — bounded wait, throws `TimeoutException` - `future.isDone()` / `future.isCancelled()` — non-blocking checks. - `future.cancel(boolean mayInterruptIfRunning)` — attempts cancellation; returns `false` if already done.

A `RunnableFuture` (via `Executors.callable(Runnable, result)`) lets you submit a `Runnable` but still get a `Future` back with a fixed result value — a useful bridge when an existing API only gives you `Runnable` S.

3. `synchronized` — Deeper Theory

`synchronized` relies on the JVM's **intrinsic lock** (a.k.a. monitor lock), one per object. Some things the article doesn't mention:

Reentrancy. Intrinsic locks are reentrant — a thread already holding a lock can re-acquire it without deadlocking itself:

```
public synchronized void outer() {
    inner(); // same thread can call this even though it's also
synchronized
}
public synchronized void inner() { }
```

Internally the JVM keeps a per-thread hold count; the lock is only released once the count returns to zero.

Lock granularity — what actually gets locked: - Instance method `synchronized void m()` → locks `this` . - Static method `synchronized static void m()` → locks the `Class` object (`MyClass.class`). - Block form `synchronized (obj) { }` → locks whatever `obj` refers to — can be any object, including a dedicated private lock object (recommended, since locking on `this` or a `Class` object exposes the lock to external code that might also synchronize on it, unintentionally).

```
private final Object lock = new Object();
public void update() {
    synchronized (lock) {
        // critical section
    }
}
```

Lock escalation in the JVM (informational, not something you control): modern JVMs use *biased locking*, *thin locks*, and *fat locks* as performance tiers — an uncontended lock is cheap; a contended one escalates to an OS-level mutex. You don't write code differently for this, but it explains why `synchronized` isn't always as slow as its reputation suggests.

`synchronized` vs `ReentrantLock` at a glance (expanded later in section 9):

	<code>synchronized</code>	<code>ReentrantLock</code>
Acquired/released	Automatically (block scope)	Manually (<code>lock()</code> / <code>unlock()</code>)
Interruptible wait	No	Yes (<code>lockInterruptibly()</code>)
Timeout support	No	Yes (<code>tryLock(timeout)</code>)
Fairness policy	No	Optional (<code>new ReentrantLock(true)</code>)
Condition variables	One implicit (<code>wait</code> / <code>notify</code>)	Multiple (<code>newCondition()</code>)

4. `wait()` / `notify()` — Deeper Theory

Why these live on `Object`, not `Thread`. Every object has an intrinsic lock and an associated **wait set** — the set of threads waiting on that object’s monitor. Since any object can be a lock, `wait()` / `notify()` had to be on `Object` itself.

The spurious wakeup problem, explained properly. The article says “use a loop,” but here’s *why*: the JVM spec permits a thread to wake from `wait()` without `notify()` / `notifyAll()` ever being called (a JVM/OS-level artifact). Additionally, even a legitimate `notifyAll()` wakes *every* waiting thread, but only one can actually proceed if the resource is single-use — the rest need to recheck and go back to waiting. Hence:

```
synchronized (lock) {
    while (!conditionIsTrue()) {    // NEVER "if" — always "while"
        lock.wait();
    }
    // proceed safely
}
```

`notify()` vs `notifyAll()`. `notify()` wakes *one arbitrary* waiting thread — cheaper, but dangerous if multiple threads are waiting for *different* conditions on the same lock (you might wake the wrong one, and it goes back to sleep, while the thread that should have woken never does — a subtle, hard-to-diagnose deadlock-like bug). Default to `notifyAll()` unless you’ve proven all waiters are interchangeable.

`IllegalMonitorStateException`. Thrown if `wait()` / `notify()` / `notifyAll()` is called without holding the object’s lock. This is why they must always be inside a `synchronized (obj)` block on that same `obj`.

Modern alternative: `java.util.concurrent.locks.Condition` (paired with `ReentrantLock`) does the same job with clearer semantics and supports multiple independent condition queues per lock:

```
Lock lock = new ReentrantLock();
Condition notEmpty = lock.newCondition();
Condition notFull = lock.newCondition();
// await() / signal() / signalAll() instead of
wait()/notify()/notifyAll()
```

5. `volatile` — Deeper Theory (Java Memory Model)

The article’s coverage (“ensures visibility, not atomicity”) is correct but shallow. Here’s the model underneath it.

The core problem `volatile` solves: without it, the compiler, JIT, and CPU are all free to reorder instructions and cache variables in registers/CPU caches, as long as *single-threaded* behavior is preserved. A second thread reading the same field may see a stale value indefinitely — not eventually, *indefinitely*, in principle.

What `volatile` actually guarantees (Java Memory Model / JLS §17.4):

- Visibility** — a write to a volatile field happens-before every subsequent read of that field by any thread. No CPU-cache staleness.
- Ordering** — the JVM/compiler cannot reorder other reads/writes around a volatile access (this is stronger than most people realize — it affects surrounding code too, not just the volatile field itself).
- No atomicity for compound actions.** `volatile int counter; counter++;` is still a read-modify-write race, because `++` is three operations (read, add, write), and `volatile` only makes each individual read or write atomic and visible — not the sequence.

“Happens-before” is the JMM’s central concept. It’s a partial ordering the JVM guarantees between actions, so you know what’s *safe* to assume about visibility across threads. Sources of happens-before edges relevant here: - A write to a `volatile` field happens-before every subsequent

read of it. - `Thread.start()` happens-before anything the started thread does. - Everything a thread does happens-before another thread's return from `Thread.join()` on it. - Unlocking a monitor happens-before any subsequent locking of that same monitor (this is what makes `synchronized` also a visibility mechanism, not just a mutual-exclusion one — a fact the article never states, and it's important).

When `volatile` is enough vs. when it's not:

Use case	<code>volatile</code> sufficient?
Boolean flag one thread sets, others read (<code>running</code> , <code>shutdownRequested</code>)	Yes
Publishing a reference to an immutable object after construction	Yes
Counter incremented by multiple threads	No — use <code>AtomicInteger</code>
"Check-then-act" logic (e.g., lazy init)	No — needs locking or double-checked locking pattern with <code>volatile</code>

Double-checked locking — the classic case where `volatile` is *required*, not optional:

```
public class Singleton {
    private static volatile Singleton instance; // volatile is
    NOT optional here
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

Without `volatile`, another thread could observe a partially-constructed `Singleton` due to instruction reordering during object construction — the reference could be set before the constructor finishes running.

6. Thread Safety — Consolidated Theory

The article scatters this across several sections. Here's the taxonomy interviewers usually expect:

Levels of thread safety (from Goetz's *Java Concurrency in Practice*, the standard reference):

1. **Immutable** — object's state can't change after construction; inherently thread-safe (e.g., `String`, `LocalDate`, records with only final fields, no setters). 2. **Unconditionally thread-safe** — mutable, but internally synchronized so callers need nothing extra (e.g., `ConcurrentHashMap`, `Vector`). 3. **Conditionally thread-safe** — safe for individual operations, but compound sequences of operations still need external locking (e.g., iterating a `synchronizedList` requires the caller to manually synchronize on it). 4. **Thread-unsafe** — no safety guarantees at all (e.g., `ArrayList`, `HashMap`).

Strategies to achieve thread safety, ranked roughly by preference: 1. **Avoid shared mutable state entirely** — thread confinement, stack-local variables, `ThreadLocal<T>`. 2. **Make it immutable** — no mutation, no race. 3. **Use existing thread-safe building blocks** — `java.util.concurrent` collections/atomics instead of hand-rolled locking. 4. **Synchronize explicitly** — `synchronized` or `Lock`, as a last resort when the above don't fit, because it's the easiest to get wrong (over-broad locks, missed critical sections, deadlocks).

`ThreadLocal<T>`, which the article names but never demonstrates:

```

private static final ThreadLocal<SimpleDateFormat> FORMATTER =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-
dd"));

// Each thread gets its own independent SimpleDateFormat instance
-
// no sharing, no synchronization needed, even though
SimpleDateFormat itself isn't thread-safe.
String formatted = FORMATTER.get().format(new Date());

```

Common use: per-thread caches, per-request context (transaction IDs, user context in web servers).
Caveat: in thread-pool environments, always call `remove()` when done, or the value leaks across pooled-thread reuse and across requests.

7. Executor Framework — Deeper Theory

The interface hierarchy (the article shows usage but not the shape):

```

Executor                                (execute(Runnable) – fire and forget)
├─ ExecutorService                    (+ submit, shutdown,
invokeAll/invokeAny)
├─ ScheduledExecutorService          (+ schedule,
scheduleAtFixedRate)

```

`ThreadPoolExecutor` is the concrete class backing most factory methods in `Executors`. Its real constructor exposes the knobs the factory methods hide:

```

new ThreadPoolExecutor(
    corePoolSize,      // threads kept alive even when idle
    maximumPoolSize,   // ceiling under load
    keepAliveTime, unit, // how long excess idle threads survive
    workQueue,         // BlockingQueue<Runnable> – buffers
    pending tasks
    threadFactory,      // for naming/priority/daemon control
    rejectedExecutionHandler // what to do when the queue AND
    pool are both full
);

```

Why experienced teams avoid the `Executors.new*` factory methods in production (a caveat the article never raises): several of them use **unbounded queues** (`newFixedThreadPool` , `newSingleThreadExecutor` both use an unbounded `LinkedBlockingQueue`) or **unbounded thread creation** (`newCachedThreadPool`), either of which can lead to `OutOfMemoryError` under sustained load. Production code typically constructs `ThreadPoolExecutor` directly with bounded queues and an explicit rejection policy.

Rejection policies (what happens when the queue is full and the pool is at max size): - `AbortPolicy` (default) — throws `RejectedExecutionException` . - `CallerRunsPolicy` — runs the task on the *calling* thread, naturally throttling the producer. - `DiscardPolicy` — silently drops the task. - `DiscardOldestPolicy` — drops the oldest queued task, then retries.

`invokeAll` vs `invokeAny` (not covered in the article at all):

```

List<Callable<Integer>> tasks = List.of(task1, task2, task3);
List<Future<Integer>> results = executor.invokeAll(tasks); //
    waits for ALL to finish
Integer first = executor.invokeAny(tasks);                //
    returns as soon as ONE succeeds, cancels the rest

```

8. Thread Pools — Sizing Theory

The article lists the pool *types* but not how to *size* them — a very common real interview question.

For CPU-bound tasks (e.g., computation, no blocking I/O), the classic guideline:

```
pool size ≈ number of CPU cores + 1
```

More threads than cores just adds context-switching overhead with no throughput gain, since cores are already saturated.

For I/O-bound tasks (waiting on network, disk, DB), threads spend most of their time blocked, not computing, so you can profitably run far more threads than cores. A commonly cited formula (Goetz):

```
pool size = number of cores × target CPU utilization × (1 + wait  
time / compute time)
```

In practice, teams usually start with something like `cores × 2` to `cores × 10` for I/O-heavy workloads and tune based on measured latency/throughput rather than deriving it analytically.

9. CompletableFuture — Full Treatment (article only namechecks this)

`CompletableFuture<T>` (Java 8+) is a `Future` that can be **composed, chained, and combined**, and can also be **completed manually** rather than only by a background task — solving the biggest pain point of plain `Future` (no way to attach a callback; `get()` always blocks).

Creating one:

```
CompletableFuture<String> cf = CompletableFuture.supplyAsync(() -  
> {  
    // runs on ForkJoinPool.commonPool() by default, or pass a  
    custom Executor  
    return fetchFromNetwork();  
});
```

Chaining transformations without blocking:

```
CompletableFuture<Integer> pipeline = CompletableFuture  
    .supplyAsync(() -> fetchUserId())  
    .thenApply(id -> id * 2)           // sync transform, same  
thread  
    .thenApplyAsync(id -> lookupScore(id)) // runs asynchronously  
    .thenApply(score -> score + 10);  
  
pipeline.thenAccept(result -> System.out.println("Final: " +  
result)); // consume, no return
```

Combining independent futures:

```
CompletableFuture<Integer> a = CompletableFuture.supplyAsync(() -  
> 10);  
CompletableFuture<Integer> b = CompletableFuture.supplyAsync(() -  
> 20);  
  
CompletableFuture<Integer> sum = a.thenCombine(b, Integer::sum);  
// waits for both, combines results
```

Sequencing dependent async calls (`thenCompose` , the async version of `flatMap`):

```
CompletableFuture<Order> order = CompletableFuture
    .supplyAsync(() -> fetchUserAsync(userId))
    .thenCompose(user -> fetchLatestOrderAsync(user)); // avoids
CompletableFuture<CompletableFuture<Order>>
```

Rule of thumb: use `thenApply` when your function returns a plain value; use `thenCompose` when your function itself returns another `CompletableFuture` .

Error handling:

```
CompletableFuture<Integer> safe = CompletableFuture
    .supplyAsync(() -> riskyCall())
    .exceptionally(ex -> { log(ex); return -1; }) // recover
from any exception
    .handle((result, ex) -> ex != null ? -1 : result); // see
both outcomes, always runs
```

Waiting on multiple futures at once:

```
CompletableFuture<Void> all = CompletableFuture.allOf(future1,
future2, future3);
all.join(); // blocks until every one completes; individual
results retrieved via each future's own get()

CompletableFuture<Object> any = CompletableFuture.anyOf(future1,
future2, future3); // first to finish wins
```

Why this matters over raw `Future` : raw `Future.get()` is always a blocking pull. `CompletableFuture` lets you build a non-blocking, event-driven pipeline — critical for reactive/async web servers, fan-out/fan-in API aggregation, and avoiding thread-pool starvation from threads sitting idle in `get()` .

10. `java.util.concurrent` — Locks, Atomics, `ConcurrentHashMap`

10a. Locks beyond `ReentrantLock`

`ReadWriteLock` / `ReentrantReadWriteLock` — separates read access (shared, many readers allowed concurrently) from write access (exclusive, one writer, no readers). Ideal for read-heavy, write-rare data structures:

```
ReadWriteLock rwLock = new ReentrantReadWriteLock();
Lock readLock = rwLock.readLock();
Lock writeLock = rwLock.writeLock();

readLock.lock();
try { return cache.get(key); } finally { readLock.unlock(); }

writeLock.lock();
try { cache.put(key, value); } finally { writeLock.unlock(); }
```

`StampedLock` (Java 8+) — adds a third mode, *optimistic reading*, which doesn't block writers at all; the reader validates afterward that no write occurred, and falls back to a real read lock only if it did. Faster than `ReentrantReadWriteLock` under heavy read contention, but more complex and non-reentrant — use only when profiling shows lock contention is a real bottleneck.

10b. Atomic classes (article names them, never demonstrates)

Atomics use **CAS (Compare-And-Swap)**, a hardware-level instruction, instead of locks — the JVM asks the CPU “if the current value is still X, atomically replace it with Y,” and retries in a loop if another thread beat it there. This is why atomics are often faster than `synchronized` under moderate contention: no thread ever blocks or sleeps.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();           // atomic ++counter, no
synchronized needed
counter.compareAndSet(5, 10);       // if current value is 5, set
to 10; returns boolean success
counter.getAndUpdate(x -> x * 2);   // atomic read-modify-write
with a lambda
```

Solves exactly the `count++` race condition the article’s `CounterExample` demonstrates, without any lock:

```
static AtomicInteger count = new AtomicInteger(0);
// threads call count.incrementAndGet() instead of count++
// Final count is reliably 20000, no race, no synchronized block
needed.
```

Other common atomics: `AtomicLong`, `AtomicBoolean`, `AtomicReference<V>` (atomic CAS on any object reference — useful for lock-free algorithms and immutable-state swaps).

Limitation: atomics only guarantee atomicity for the *single* variable they wrap. Coordinating multiple related atomics still needs a lock if you need cross-variable consistency.

10c. `ConcurrentHashMap` internals (article names it, never explains it)

Unlike `Collections.synchronizedMap(new HashMap<>())`, which locks the *entire map* on every operation (one big lock — a bottleneck under concurrent access), `ConcurrentHashMap` uses much finer-grained internal locking: - Historically (Java 7 and earlier): **segment-level locking** — the map was divided into segments, each independently lockable, so concurrent writers to different segments didn’t block each other. - Java 8+: segments were dropped in favor of **per-bin (per-bucket) locking** using `synchronized` on individual bin nodes plus CAS operations, giving even finer granularity. - **Reads are (mostly) lock-free** — `get()` doesn’t block at all in the common case, using `volatile`-backed reads of internal nodes.

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>
();
map.put("a", 1);
map.computeIfAbsent("b", k -> 2);           // atomic "insert if
missing"
map.merge("a", 1, Integer::sum);           // atomic "update or
insert"
map.compute("a", (k, v) -> v == null ? 1 : v + 1); // atomic
conditional update
```

Important caveat: `ConcurrentHashMap`’s *individual* operations are atomic, but compound sequences (e.g., “check size, then act”) across multiple calls are **not** atomic as a whole unless you use its atomic compound methods (`computeIfAbsent`, `merge`, `compute`) — same conditional-safety caveat as category 3 in the thread-safety taxonomy above.

Also, unlike `Hashtable` / `synchronizedMap`, `ConcurrentHashMap` does **not** allow `null` keys or values — this trips people up because plain `HashMap` does.

11. Deadlocks, Livelocks, and Starvation

The article covers deadlocks well but never mentions livelock or starvation — both common in interviews as “what’s the difference” questions.

Deadlock — threads are stuck, each waiting on a resource the other holds; nothing moves. Classic four necessary conditions (Coffman conditions) — all four must hold simultaneously for deadlock to occur: 1. Mutual exclusion — resource can’t be shared. 2. Hold and wait — thread holds one resource

while waiting for another. 3. No preemption — a resource can't be forcibly taken from a thread. 4. Circular wait — a cycle of threads each waiting on the next.

Break any one condition and deadlock becomes impossible — this is *why* consistent lock ordering works (it eliminates circular wait).

Livelock — threads are *not* blocked, they're actively running, but they make no real progress because they keep responding to each other in a way that undoes their own progress. Classic analogy: two people in a hallway, both stepping aside to let the other pass, both stepping the same direction repeatedly, forever. In code, this often happens with overly "polite" retry/backoff logic:

```
// Simplified livelock pattern: two threads each release-and-
retry
// when they detect contention, but with identical, deterministic
timing,
// they perpetually re-collide.
while (!tryAcquireBoth(lockA, lockB)) {
    releaseAny();
    // no randomized backoff -> threads stay in lockstep forever
}
```

Fix: add randomized/exponential backoff so retry timings desynchronize, or use ordered lock acquisition instead of a retry loop at all.

Starvation — a thread *can* eventually run, but effectively never gets scheduled/gets the lock because other threads keep monopolizing the resource (e.g., always losing to higher-priority threads, or an unfair lock where the same threads keep winning the race). Unlike deadlock/livelock, the system as a whole is making progress — just not for that one thread. **Fix:** fair locks (`new ReentrantLock(true)`), bounded/round-robin scheduling, avoiding thread priority tricks.

	Threads blocked?	System making progress?	Typical fix
Deadlock	Yes	No	Lock ordering, timeouts
Livelock	No (actively running)	No	Randomized backoff
Starvation	No (just unlucky)	Yes, overall	Fair locks/scheduling

12. Coding Questions

Q1. Fix the race condition

The following is not thread-safe. Fix it two different ways.

```
public class Counter {
    private int count = 0;
    public void increment() { count++; }
    public int get() { return count; }
}
```

Answer:

Fix 1 — synchronized:

```
public class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public synchronized int get() { return count; }
}
```

Fix 2 — AtomicInteger (preferred, lock-free, generally faster under contention):

```
public class Counter {  
    private final AtomicInteger count = new AtomicInteger(0);  
    public void increment() { count.incrementAndGet(); }  
    public int get() { return count.get(); }  
}
```

Note that `get()` also needs synchronization/atomicity in Fix 1 — a common mistake is synchronizing only the writer and assuming an unsynchronized reader is safe. It isn't: without synchronization on the read side too, there's no happens-before edge guaranteeing the reader sees the latest value.

Q2. What's wrong with this singleton, and fix it

```
public class Singleton {  
    private static Singleton instance;  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Answer: Not thread-safe — two threads can both pass the `null` check simultaneously and each construct a separate instance (or, worse, one thread returns a reference to a partially-constructed object). Three valid fixes, in order of general preference:

```

// 1. Simplest: eager initialization (no laziness, but always
correct and simple)
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    public static Singleton getInstance() { return INSTANCE; }
}

// 2. Lazy holder idiom (lazy AND simple, relies on classloader
guarantees)
public class Singleton {
    private static class Holder {
        static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() { return
Holder.INSTANCE; }
}

// 3. Double-checked locking (lazy, but needs volatile – see
Section 5)
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}

```

Q3. Implement a producer-consumer with `BlockingQueue` (no manual wait/notify)

Answer:

```

import java.util.concurrent.*;

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new LinkedBlockingQueue<>
(10); // bounded, capacity 10

        Runnable producer = () -> {
            try {
                for (int i = 0; i < 20; i++) {
                    queue.put(i); // blocks if full, no manual
wait() needed

                    System.out.println("Produced: " + i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        };

        Runnable consumer = () -> {
            try {
                for (int i = 0; i < 20; i++) {
                    Integer item = queue.take(); // blocks if
empty

                    System.out.println("Consumed: " + item);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        };

        ExecutorService executor =
Executors.newFixedThreadPool(2);
        executor.submit(producer);
        executor.submit(consumer);
        executor.shutdown();
    }
}

```

`BlockingQueue` (`put()` / `take()`) internally handles all the locking/ wait / notify machinery that the article's manual `SharedData` example implements by hand — this is the idiomatic, modern way to write producer-consumer code.

Q4. Given this code, will it deadlock? Why or why not?

```

Object lockA = new Object();
Object lockB = new Object();

Thread t1 = new Thread(() -> {
    synchronized (lockA) {
        synchronized (lockB) { /* work */ }
    }
});
Thread t2 = new Thread(() -> {
    synchronized (lockA) {
        synchronized (lockB) { /* work */ }
    }
});

```

Answer: No deadlock. Both threads acquire the locks in the **same order** (`lockA` then `lockB`). One thread will simply block briefly waiting for the other to finish and release `lockA` , then proceed normally. Deadlock requires a *circular* wait — this would only deadlock if `t2` acquired `lockB` before `lockA` (inconsistent ordering).

Q5. Write a thread pool that processes tasks and correctly waits for all of them before printing a final result

Answer:

```

import java.util.concurrent.*;
import java.util.*;

public class SumAllTasks {
    public static void main(String[] args) throws
InterruptedException, ExecutionException {
        ExecutorService executor =
Executors.newFixedThreadPool(4);

        List<Callable<Integer>> tasks = new ArrayList<>();
        for (int i = 1; i <= 10; i++) {
            int n = i;
            tasks.add(() -> n * n); // square each number
        }

        List<Future<Integer>> futures =
executor.invokeAll(tasks); // waits for all to complete

        int sum = 0;
        for (Future<Integer> f : futures) {
            sum += f.get(); // no need to block-wait manually,
// invokeAll already ensured completion
        }
        System.out.println("Sum of squares: " + sum);

        executor.shutdown();
    }
}

```

Q6. What does this print, and why is the output unreliable?

```

public class VisibilityDemo {
    private static boolean flag = false; // note: NOT volatile

    public static void main(String[] args) throws
InterruptedException {
        Thread reader = new Thread(() -> {
            while (!flag) { /* spin */ }
            System.out.println("Flag detected!");
        });
        reader.start();
        Thread.sleep(100);
        flag = true;
        System.out.println("Flag set.");
    }
}

```

Answer: In principle this can loop **forever** — the reader thread may cache `flag` in a register/CPU cache and never observe the write from `main`, since there's no happens-before relationship between the write and the read (no `volatile`, no lock, no other synchronization). In practice, on many JVM/hardware combos it'll often "work" for a simple demo like this, which is exactly what makes this bug dangerous — it's not deterministic, and can pass in dev and fail in production under different JIT optimization levels or hardware. Fix: declare `flag` as `volatile`.

Q7. Implement a simple `CompletableFuture` pipeline: fetch a user, then fetch their orders, then count them — all asynchronously

Answer:

```

import java.util.concurrent.CompletableFuture;
import java.util.List;

public class AsyncPipeline {
    record User(int id, String name) {}

    static CompletableFuture<User> fetchUser(int id) {
        return CompletableFuture.supplyAsync(() -> {
            // simulate network call
            return new User(id, "Alice");
        });
    }

    static CompletableFuture<List<String>> fetchOrders(User user)
{
        return CompletableFuture.supplyAsync(() ->
List.of("order1", "order2", "order3"));
    }

    public static void main(String[] args) {
        CompletableFuture<Integer> orderCount = fetchUser(1)
            .thenCompose(AsyncPipeline::fetchOrders) // async
call depends on prior result
            .thenApply(List::size) // simple
sync transform
            .exceptionally(ex -> {
                System.err.println("Failed: " + ex.getMessage());
                return -1;
            });

        System.out.println("Order count: " + orderCount.join());
    }
}

```

Q8. Explain why this reader/writer pattern is broken, and fix it with

`ReadWriteLock`

```

public class Cache {
    private final Map<String, String> map = new HashMap<>();
    public String get(String key) { return map.get(key); }
    // unsynchronized read
    public void put(String key, String value) { synchronized
(this) { map.put(key, value); } } // synced write
}

```

Answer: `HashMap` is not thread-safe for concurrent read+write, even if only writes are synchronized — a reader can observe a `HashMap` in a structurally inconsistent state mid-resize (in the worst case, this can even cause an infinite loop in old JDKs). Every access — reads included — needs to go through the same lock, or use a genuinely concurrent structure.

```

public class Cache {
    private final Map<String, String> map = new HashMap<>();
    private final ReadWriteLock lock = new
ReentrantReadWriteLock();

    public String get(String key) {
        lock.readLock().lock();
        try { return map.get(key); }
        finally { lock.readLock().unlock(); }
    }
    public void put(String key, String value) {
        lock.writeLock().lock();
        try { map.put(key, value); }
        finally { lock.writeLock().unlock(); }
    }
}

```

Or, more simply, just use `ConcurrentHashMap` directly and skip manual locking entirely — the right call unless you specifically need custom multi-key transactional semantics that `ConcurrentHashMap`'s atomic methods can't express.

Q9. Diagnose the bug: this `notify()` usage can lose a signal

```

class Signal {
    private boolean ready = false;
    public void await() throws InterruptedException {
        if (!ready) {                // BUG: "if" instead of
"while"
            wait();
        }
    }
    public void signal() {
        synchronized (this) {
            ready = true;
            notify();
        }
    }
}

```

Wait — spot a second bug too. **Answer:** Two bugs: 1. `if (!ready) wait();` should be `while (!ready) wait();` — guards against spurious wakeups (Section 4). If the thread wakes spuriously before `ready` is actually true, `if` lets it proceed incorrectly. 2. `await()` itself isn't `synchronized` and doesn't hold the lock around `wait()` — this throws `IllegalMonitorStateException` at runtime, since `wait()` requires the calling thread to already hold the monitor for `this`.

Fixed version:

```

class Signal {
    private boolean ready = false;
    public synchronized void await() throws InterruptedException
    {
        while (!ready) {
            wait();
        }
    }
    public synchronized void signal() {
        ready = true;
        notifyAll(); // notifyAll, not notify, in case multiple
waiters exist
    }
}

```

Q10. Given `newFixedThreadPool(2)` and 5 blocking tasks, trace what happens

```

ExecutorService executor = Executors.newFixedThreadPool(2);
for (int i = 1; i <= 5; i++) {
    int taskId = i;
    executor.submit(() -> {
        System.out.println("Task " + taskId + " running on " +
Thread.currentThread().getName());
        try { Thread.sleep(1000); } catch (InterruptedException
e) {}
    });
}
executor.shutdown();

```

Answer: Only 2 threads exist in the pool, so only 2 tasks run concurrently at any time. Tasks 1 and 2 start immediately; tasks 3, 4, 5 sit in the pool's internal unbounded `LinkedBlockingQueue` until a thread frees up (roughly 1 second apart, given the `sleep(1000)`). Total wall-clock time $\approx$ 3 seconds (5 tasks / 2 concurrent $\approx$ 2.5, rounded up to 3 full "waves"). `shutdown()` doesn't cancel anything in progress or queued — it just stops the executor from accepting *new* tasks; already-submitted tasks still run to completion. To actually observe the queueing, you'd see task IDs 1 and 2 print immediately, then 3 and 4 roughly a second later, then 5 a second after that.

Quick-Reference Summary Table

Topic	One-line takeaway
Thread lifecycle	6 real JVM states; <code>BLOCKED</code> $\neq$ <code>WAITING</code>
Runnable/Callable	<code>Callable</code> returns a value + can throw checked exceptions, only usable via <code>ExecutorService</code>
<code>synchronized</code>	Reentrant intrinsic lock; also a visibility mechanism, not just mutual exclusion
<code>wait</code> / <code>notify</code>	Always in a <code>while</code> loop, always inside <code>synchronized</code> on the same object; prefer <code>notifyAll</code>
<code>volatile</code>	Guarantees visibility + ordering, never atomicity for compound ops
Thread safety	Prefer immutability > existing concurrent utilities > manual locking
Executor framework	<code>Executor</code> $\rightarrow$ <code>ExecutorService</code> $\rightarrow$ <code>ScheduledExecutorService</code> ; avoid unbounded default factories in production
Thread pools	CPU-bound $\approx$ <code>cores+1</code> ; I/O-bound scales much higher
<code>CompletableFuture</code>	Non-blocking, composable; <code>thenApply</code> for sync transforms, <code>thenCompose</code> for chained async calls
Locks/atomics/ <code>ConcurrentHashMap</code>	CAS-based atomics are lock-free; <code>ConcurrentHashMap</code> uses fine-grained per-bin locking, not one big lock
Deadlock/livelock/starvation	Deadlock = blocked forever; livelock = running but no progress; starvation = unlucky but system still progresses